
SYNC_CONTRACT.md

Canonical Synchronization Contract for the Lattice Vault

1. Purpose

Establish a governed, deterministic synchronization protocol ensuring that every environment bound to the Lattice maintains perfect lineage alignment with the canonical origin repository.

The contract prevents:

- Drift
- Silent edits
- Forked realities
- Non-governed mutations

2. Scope

This contract governs synchronization of:

- Governance layer (Constitution, Operator Manual, Playbook)
- Unified Spec
- Runtime
- Pipeline
- Epistemic substrate
- Exports
- All canonical documents referenced by VAULT_INDEX.md

Excluded:

- Secrets
- Local caches
- Ephemeral scratch files

3. Roles

3.1 ORIGIN

Definition:

The single canonical source of truth.

Instance:

github.com/kushkillerjsixx66/canonical-vault (main branch)

Authorities:

- Owns canonical lineage
- Owns canonical tags
- Owns governance state

3.2 NODE

Definition:

Any environment that reads from or writes to the Vault.

Examples:

Termux device, laptop, cloud VM, AI workspace.

Authorities:

- May propose changes via branches
- Must never mutate main directly

3.3 AGENT

Definition:

Automated or AI-driven processes interacting with the Vault.

Authorities:

- Default: read-only
- Write access requires explicit Agent Policy File

4. Invariants

Invariant 1 — Single Canonical Branch
main is the only branch treated as truth.

Invariant 2 — No Out-of-Band Edits

All changes must be:

- Committed
- Diffable
- Attributable

- Traceable to an intent

Invariant 3 — Sync Before Write

No environment may push without a successful pull from ORIGIN/main.

Invariant 4 — Index Supremacy

VAULT_INDEX.md defines the canonical file graph.

Any canonical file must be reachable from it.

5. Environment Classes

Class A — Authoring Nodes

Full authoring capability.

May create branches and PRs.

Class B — Runtime Nodes

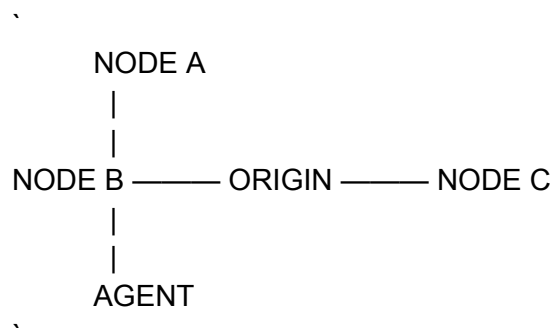
Read-only except for local runtime execution.

Class C — Ephemeral Sandboxes

Shallow clones, no pushes, no persistence.

6. Sync Topology

All synchronization flows follow a star topology:



No peer-to-peer sync.

All flows route through ORIGIN.

7. Operational Procedures

7.1 Binding a New Node

1. Clone ORIGIN
2. Verify clean state
3. Confirm VAULT_INDEX.md integrity
4. Register node (optional)

7.2 Standard Work Session (Class A)

Before work:

- Pull from ORIGIN
- Create feature branch

During work:

- Commit with intent
- Maintain clean diffs

After work:

- Rebase onto updated main
- Push branch
- Open PR

7.3 Runtime Sync (Class B/C)

- pull
- reset --hard origin/main
- Optionally checkout tagged releases

No commits.

No pushes.

8. Agent Write Policy

Default:

Agents are read-only.

To grant write access:

1. Create 00governance/agentpolicies/<agent_id>.md

2. Define:

- Allowed paths
- Allowed operations
- Required commit tags

3. Agent writes only to agent/<id>/<intent> branches

4. Human operator merges via PR

9. Drift Detection & Resolution

Detection:

- Local changes when none expected
- Divergent commit history
- Non-fast-forward pulls

Resolution:

Intentional changes:

- Move to feature branch
- Reset main to ORIGIN

Accidental changes:

- Hard reset to ORIGIN

Divergence on ORIGIN:

- Investigate
- Rebase or merge
- Document decision

10. Compliance Checks

- Clean working tree on runtime nodes
- No direct commits to main
- Tag discipline enforced
- Index consistency maintained

11. Violations

Any violation of this contract constitutes lineage corruption and must be:

1. Logged

2. Investigated

3. Reconciled via governed amendment

12. Amendment Protocol

This contract may only be modified via:

- A dedicated branch
- A PR referencing governance intent
- Operator approval
- Version bump
